第1章 对象式程序设计基本概念

教学目标：

掌握面向对象软件开发的思想要点

1.1 面向过程与面向对象程序设计

- 几个基本事实
  - 认识、分析和解决问题应采用与问题复杂程度相匹配的方法论：还原论→系统论；
  - 认识、分析和解决复杂问题应采用抽象程度高的方法；
    - 现代建筑利用构件来建造
    - 成功的计算机制造商并不生产所有配件
    - 认识复杂事物采用高度抽象的概念
  - 硬件价格不断下降、性能越来越高、应用越来越广、解决的问题越来越复杂；

- 几个基本事实（续）
  - 程序设计语言从面向机器到面向过程再到面向对象的发展历程反映了程序越来越复杂的趋势，也越来越接近人的思维习惯；
  - 现代软件的本质特征是其固有的复杂性，这是软件危机产生的根源；
  - 应对和控制复杂性的主要方法有：分解、抽象、模块化和信息隐藏等。

- 面向过程程序设计存在的问题
  - 各种类型的数据与过程间的联系错综复杂，软件的可靠性、可读性和可维护性差；
  - 设计方法强烈依赖系统逻辑模型的完整性描述，难以作到系统的动态变化与扩展，系统开发速度慢、周期长、费用高；
  - 自顶而下的瀑布式的设计，使数据结构的设计仅服从于当前功能实现的要求，为以后的扩充和重用设置了巨大的障碍。

- 两者的抽象程度不同
  - 面向过程的程序：算法+数据结构=程序， C, Fortran
  - 面向对象的程序：对象及其相互作用、协调, C++, Java
- 两者的方法论不同：

面向过程的程序设计：

还原论 → 面向过程的问题求解 → 面向过程的程序设计

↓ ↓ ↓

难以认识复杂事物 难以解决复杂问题 难以设计复杂程序

面向对象的程序设计：

系统论 → 面向对象的问题求解 → 面向对象的程序设计

↓ ↓ ↓

便于认识复杂事物　便于解决复杂问题　便于设计复杂程序

- 两者并不矛盾
  - 前者注重局部，后者注重整体（各要素之间的相互关系）；
  - 前者适用于设计简单、规模小的程序，后者适用于设计复杂、规模大的程序；
  - 在面向对象程序设计的局部仍需要面向过程的程序设计

1.2 面向对象程序设计的思想要点

- 程序由对象组成，这些对象首先是自治、自恰的，同时它们还相互通信、协调和配合，从而共同完成整个程序的任务和功能。
  - 对象：以数据为中心，是集成了数据和其上的操作独立、自恰（通过自身定义的操作来管理自己）的逻辑单位；
  - 对象具有唯一的标识和固定的对外接口，接口是对象与外界通信的通道；
  - 对象的属性表示它所处的状态（静态特征），操作用来改变对象状态达到特定功能（动态特征）；
  - 对象是现实世界实体或概念在计算机中的抽象表示。

一个电视机对象

- 对象的关系
  - 继承关系(is-a)：当对象A是对象B的特例时，称对象A继承了对象B。例：电视机、黑白电视机、彩色电视机。
  - 合成关系(has-a)：当对象A的引用是对象B的属性时，称对象A和对象B之间是合成关系。
- 对象的引用是指对象的名称、地址、句柄等可以获取或操纵该对象的途径。对象的引用所占用的内存空间要少得多，它只是找到对象的一条线索，通过它，程序可以找到真正的对象，访问这个对象的数据，调用这个对象的方法。
  - 依赖关系(use-a)：如果一个对象需要通过消息（方法调用）与另一个对象交互，这个对象就依赖于另一个对象。

- 类是同种对象的集合与抽象，对象是类的实例，类是产生对象的模板。类可以派生子类，子类继承超类的属性和方法。
- 在面向对象程序设计方法中，其程序结构是一个类的集合和各类之间以继承关系联系起来的结构，有一个主程序，在主程序中创建各对象并规定它们之间传递消息的规律。从程序执行角度来看，可以归结为各对象和它们之间的消息通信（方法调用）。面向对象程序设计最主要的特征是各对象之间的消息传递和各类之间的继承。
- 类型是一组值域及作用其上的一组操作的抽象。

1.3 面向对象软件开发过程

- OOA(object oriented analysis)：明确需求，从问题中抽取对象模型（概念模型）。主要解决“做什么”的问题。
- OOD(object oriented design)：将模型细化，设计类，包括类的属性、操作和类间关系，同时考察是否有直接可以引用的已有类或部件。主要解决“怎么做”的问题。
- OOP(object oriented programming)：选定一种面向对象的程序设计语言，具体编码实现上一阶段类的设计，并在开发过程中引入测试，完善整个解决方

1.4 面向对象程序设计的主要特点

- 封装性(Encapsulation)：把表示状态的属性和对状态的操作包装成一个对象类型，使得对状态的存取只能通过封装提供的接口进行。
  - 良好的封装能隐藏大量细节；
    - 信息隐藏
    - 实现隐藏
  - 在对象封装基础上，对象具有保持自己状态的能力；

- 继承性(Inheritance)：在已有类的基础上定义和实现新类，继承能有效地支持类的复用，在保证质量的前提下提高了开发效率，
  - 当需要在系统中增加新特征时，所需的新代码最少；
  - 当继承和多态相结合使用时，为修改系统所需变动的源代码最少。

- 多态性(Polymorphism)：表现为类型和操作具有多种形态，
  - 类型的多态：如果A是B的子类，则A的对象也是B的对象；
  - 操作的多态：
    - 子类对父类方法的改写(override)，也称为覆盖；
    - 在同一个类中定义多个同名的不同方法，称为名复用(overload)，也称为重载。

- 关于多态性的进一步说明：
  - 是OOP中功能最强大的特性；
  - 类型的多态导致对象引用变量具有静态类型和动态类型；
  - 多态的意思是具有多种形态，在面向对象程序设计中意味着通过动态绑定原理，使用单个变量来引用不同类型的对象，自动调用引用对象类的对应方法；
  - 绑定通常是指变量与其值、过程调用与相应可执行代码的确定和关连。能在编译时期完成的绑定称静态绑定，反之，则称动态绑定；
  - 多态性与继承性密切相关

1.5 面向对象技术的优点

- 面向对象技术更充分体现了分解、抽象、模块化和信息隐藏等思想，实现了对“封状性、继承性和多态性”的 语言级直接支持，可以有效（更容易）提高软件生产率、缩短开发时间、提高软件质量，是控制软件复杂性的有效途径。
- 在面向对象方法中，分析和设计阶段采用一致的概念和表示法。

1.6 本章小结

- 面向对象的软件开发就是按照对象的观点对所要构建的软件进行分析、设计和编码实现，对于复杂软件系统，良好的分析和设计是成功的关键。
- 按照对象的观点，程序由对象组成，这些对象首先是自治、自恰的，同时它们还相互通信、协调和配合，从而共同完成整个程序的任务和功能。
- 类是同种对象的集合与抽象，对象是类的实例，类是产生对象的模板。类可以派生子类，子类继承超类的属性和方法。

- 在面向对象程序设计方法中，其程序结构是一个类的集合和各类之间以继承关系联系起来的结构，有一个主程序，在主程序中创建各对象并规定它们之间传递消息的规律。
- 面向对象程序设计的主要特点是封状性、继承性和多态性。
- 面向对象程序设计能够满足现代软件系统对可重用性、可扩充性和可管理性提出的苛刻要求，特别适合大规模、复杂软件的标准化生产过程。